**QBYTE STUDIO**

# Plan de Migración

Versión: 1

Fecha: 15/10/2026

## Control de Cambios

| Versión | Causa del Cambio | Responsable del Cambio | Fecha del Cambio |
|---|---|---|---|
| 1 | Versión inicial | Adrian Delgado | 15/10/2026 |
| 2 | versión inicial | Dilan Muñoz | 15/10/2026 |

**Tabla de contenido**


Introducción 4

Objeto 4

Alcance 4

Razones para la Migración de Datos 5

Base de Datos Origen y Posible Base de Datos Destino 6

Tipo de Migración 6

Metodología de la Migración de Datos 7

Planeación 7

Migración 8

Validación de datos 8

Técnica de Migración de Datos 9

Extracción 9

Diccionario de Datos de la Base de Datos Origen 9

Mapeo de Campos 9

Transformación 9

Carga 9

Validación 9

Estrategia de Migración de Datos 10

Lista de Referencias 11

# Introducción

## Objeto

El propósito de este documento es definir el procedimiento técnico, las estrategias y las técnicas necesarias para migrar la base de datos del proyecto QBYTE Studio desde un entorno de desarrollo local  hacia un entorno de hosting en la nube (MySQL en PythonAnywhere). Esto garantizará que la aplicación esté disponible para los usuarios finales de forma segura, escalable y confiable.

## Alcance

El presente Plan de Migración está dirigido a:

El equipo de desarrollo de QBYT Studio (Adrián Delgado y Dilan Muñoz), quienes ejecutarán los pasos técnicos de la migración.

El instructor del SENA, quien supervisará y evaluará el cumplimiento del procedimiento documentado.

Futuros desarrolladores o administradores del proyecto, podrán usar este documento como guía de referencia para realizar migraciones similares o mantener la base de datos en PythonAnywhere.

## Razones para la Migración de Datos

La migración es necesaria porque actualmente la base de datos de QBYTE Studio reside en un entorno local, lo que limita el acceso a la aplicación únicamente desde el equipo de desarrollo. Al migrar a PythonAnywhere:

**Disponibilidad 24/7**: Los usuarios podrán acceder a la plataforma desde cualquier lugar.

**Escalabilidad**: Permite manejar múltiples usuarios concurrentes (docentes y estudiantes).

**Colaboración**: El equipo puede trabajar de forma remota sin depender de una máquina local.

**Respaldo y seguridad**: PythonAnywhere ofrece servicios gestionados con respaldos automáticos.

**Base de Datos Origen y Posible Base de Datos Destino**

| Característica | Origen (Local - XAMPP) | Destino (Cloud - PythonAnywhere) |
|---|---|---|
| **Motor de BD** | MySQL | MySQL |
| **Herramienta de Administración** | phpMyAdmin | Consola MySQL / phpMyAdmin (en planes pagos) |
| **Ventajas** | Fácil de usar, interfaz gráfica intuitiva, viene preconfigurado. | Totalmente gestionado, escalable, disponible 24/7, incluye respaldos automáticos. |
| **Desventajas** | Limitado a la red local, no es accesible de forma remota por defecto. | Requiere configuración manual de credenciales y pasos extra para conexiones remotas. |

**Tipo de Migración**

Es una migración HOMOGÉNEA, porque tanto el origen (XAMPP) como el destino (PythonAnywhere) utilizan el mismo motor de base de datos: MySQL. No hay cambios en el gestor, solo en la ubicación y el entorno Es una migración HOMOGÉNEA, porque tanto el origen (XAMPP) como el destino (PythonAnywhere) utilizan el mismo motor de base de datos: MySQL. No hay cambios en el gestor, solo en la ubicación y el entorno

## Metodología de la Migración de Datos

### Planeación

*La planeación de la migración de datos para QBYTE STUDIO establece las directrices, fases y controles técnicos necesarios para asegurar la transferencia de la información y la estructuración de la base de datos hacia el servidor de PythonAnywhere.*

| | | | | | |
|---|---|---|---|---|---|
| **Planeación** | Alcance, estrategias y fases | Identificar requerimientos | Análisis de riesgos | Análisis de la situación actual y final | Determinar la factibilidad técnica y económica |
| **Aprobación del plan** | Revisión del plan con los involucrado | Obtención del visto bueno del plan | | | |
| **Análisis del esquema actual y del nuevo esquema** | Análisis de integridad | Análisis de consistencia | Análisis de valores nulos y tipos de dato | Verificar el tamaño de los objetos y de la base de datos | Verificar las características de la base de datos |
| **Selección y/o desarrollo de la aplicación para migrar** | Evaluación de las alternativas | Compra o desarrollo de la aplicación | | | |
| **Pruebas** | Pruebas del proceso de migración | Pruebas de las aplicaciones que usarán la base de datos migrada | | | |

***Planeación**: el alcance, estrategias y fases define el ciclo completo para trasladar o inicializar las tablas de usuarios (docentes, estudiantes), gestión de clases, ejercicios auditivos y módulos de mensajería desde el entorno de desarrollo local hacia el servidor de producción en la nube de PythonAnywhere.*

***Identificar requerimientos:*** *Determinar los volúmenes de datos iniciales, los esquemas de tablas relacionales en MySQL en Pythonanywhere y las dependencias de claves foráneas requeridas.*

***Análisis de riesgos:*** *Identificación de posibles inconvenientes durante el proceso, limites de conexiones o recursos propios de las cuentas alojadas en PythonAnywhere. Duplicidad de registros en los perfiles de usuarios. Mitigación planificar la realización de respaldos previos mediante archivos y pruebas de despliegue en la consola de PythonAnywhere.*

***Análisis de la situación actual y final:*** *Evaluar el estado inicial de la información de prueba en el entorno local frente a la estructura normalizada final alojada en el servidor web de PythonAnywhere*

***Determinar la factibilidad técnica y económica****: Validar que las herramientas seleccionadas sean de código abierto (Python, Django, MySQL) y que el plan gratuito o de pago de PythonAnywhere soporte adecuadamente la carga operativa del sistema sin costos excesivos.*

***Aprobación del plan****: Revisión del plan con los involucrados socializando las ventanas de despliegue y migración con el equipo de desarrollo. Obtener el visto bueno del plan validando y dando aceptación final por parte de los líderes del proyecto para proceder con la ejecución técnica del servidor.*

***Análisis del esquema actual y del nuevo esquema:*** *En el análisis de integridad verificar que las llaves primarias y foráneas se mantengan estrictamente intactas tras la importación en PythonAnywhere para evitar registros huérfanos.*

***Análisis de consistencia****: Comprobar que los datos cumplan con las restricciones de negocio (ej. unicidad en los correos electrónicos de los usuarios y validación de campos obligatorios).*

***Analisis de valores nulos y tipos de datos****: Revisar que campos permiten valores vacios (null=True) en los modelos de Django y cuales son estrictamente obligatorios (blank=false), asegurando la correspondencia exacta de tipos (VARCHAR, INT, DATETIME) en MySQL.*

***Verificar el tamaño de los objetos y de la base de datos****: Medir el volumen estimado de las tablas operativas para garantizar que se ajusten a los límites de almacenamiento de bases de datos permitidos por el plan de PythonAnywhere.*

***Selección y/o desarrollo de la aplicación para migrar****: La evaluación de las alternativas y el uso de respaldos SQL directos mediante phpmyAdmin (o la pestaña Databases de PythonAnywhere) frente a las herramientas nativas de migracion de Django (python manage.py migrate).*

***Desarrollo de la aplicación****: Se optó por utilizar la combinación de las migraciones nativas de Django (migrate) junto con la importación de archivos de respaldo .sql a través de la interfaz de MySQL en PythonAnywhere, adaptándose perfectamente a los modelos de la plataforma (clase, docente, ejercicios, estudiante, mensajería).*

***Pruebas:*** *Ejecución simulada de los comandos de migración a través de la pestaña de consola (bash) de PythonAnywhere para validar que los datos transiten sin errores de sintaxis o de integridad nacional.*

**Migración**

*La fase de migración comprende la ejecución controlada y secuencial de los procedimientos técnicos destinados a consolidar la información en la base de datos MySQL de producción en PythonAnywhere*

| | | | |
|---|---|---|---|
| **Migración** | Respaldo de la base de datos origen | Extracción, transformación y carga | Respaldo de la base de datos destino |

***Respaldo de la base de datos origen****: Generación previa de un archivo de respaldo local (.sql) que garantice un punto de restauración seguro antes de cualquier manipulación de datos.*

***Extracción, transformación y carga (ETP/ETL)****: Proceso sistemático de exportación de los datos locales, su adaptación al juego de caracteres compatible con PythonAnywhere y su posterior importación en la base de datos MySQL asignada en la plataforma.*

***Respaldo de la base de datos destino:*** *Creación de una nueva copia de seguridad (exportación .sql desde PythonAnywhere) inmediatamente después de culminar con éxito la carga de datos en el servidor.*

**Validación de datos**

*Una vez finalizada la carga de información en el servidor, se realizan verificaciones cuantitativas y cualitativas para certificar la fidelidad y exactitud del proceso.*

| | | | |
|---|---|---|---|
| **Resultados** | Medición de resultados | Análisis de resultados | |
| **Cierre del proyecto** | Reporte final | Resumen ejecutivo | Memoria técnica |

**Técnica de Migración de Datos**

**Extracción**

**Diccionario de Datos de la Base de Datos Origen**

Para la puesta en marcha de QBYTE Studio en PythonAnywhere, el sistema extrae la estructura a partir de los modelos relacionales principales definidos en Django:

**Tablas Maestras (independientes)**: auth_user, docente, estudiante (almacenan la información esencial de los usuarios del sistema).

**Tablas dependientes (Relacionadas):** clase, ejercicios, mensajería (dependen directamente de las llaves foráneas de los usuarios y aulas creadas).

Determinar cuáles son las tablas maestras y las dependientes.

***Mapeo de Campos***

*Mapeo de campos desde el origen al posible destino.*

*Ejemplo:*




| Tabla Productos | | | |
|---|---|---|---|
| **Campo** | **Tipo de dato SQL Server** | **Campo** | **Tipo de datos MySQL** |
| Id_producto | INT IDENTITY PRIMARY KEY | | INT AUTO_INCREMENT PRIMARY KEY |
| nombre | VARCHAR (100) | | VARCHAR |
| color | VARCHAR (15) CHECK (IN) | | ENUM |
| precio | INT | | INT |
| Stock | NUMERIC (10,3) | | DECIMAL (10,3) |

Dado que tanto el origen (XAMPP/MySQL) como el destino (PythonAnywhere/MySQL) utilizan el mismo motor de base de datos relacional, el mapeo es directo y homogéneo, sin requerir transformaciones complejas de datos.

**Transformación**

Se documenta el procedimiento paso a paso para la depuración y acondicionamiento de los datos previos al despliegue en PythonAnywhere:

Limpieza de datos: Eliminación de espacios en blacno excedentes en los campos de texto correspondientes a nombres de clases y correos electrónicos.

**Estandarización de codificación:** Asegurar que los scripts de datos utilicen codificación utf-8 para prevenir errores con tildes y caracteres especiales en el servidor web.

**Validación de llaves foráneas:** Verificación previa de que los IDs referenciados de los docentes existan en la base de datos antes de asociales registros en las tablas de clases o ejercicios.

**Carga**

La importación y persistencia de los datos estructurados en PythonAnywhere se realiza ejecutando las migraciones mediante la consola bash integrada de la plataforma:

*Python manage.py makemigrations*
*Python manage.py migrate*

y para la carga de datos masivos iniciales, se realiza la importación del archivo .sql optimizado utilizando la sección de bases de datos o ejecutando consultas directas en la consola MySQL de PythonAnywhere.

**Validación**

Para comprobar de manera cuantitativa el exito del proceso de transferencia y el registro integro de la información en el servidor de PythonAnywhere se implementa una consulta SQL de verificación utilizando una variable de control:

*SET @total_registro = (SELECT COUNT(*) FROM qbyte_app_clase);*

*SELEC @total_registros AS Total_Clases_Migradas;*

**Estrategia de Migración de Datos**

El tipo de migración de datos es Big bang (Migración en ventana de mantenimiento única).

Teniendo en cuenta que QBYTE STUDIO se despliega en PythonAnywhere como una plataforma académica controlada para instituciones, y que su volumen inicial de datos se estructura de manera planificada antes de poner en marcha URL pública del servicio, la estrategia de Big Bang resulta la ideal.

Permite realizar el volcado completo de la base de datos de MySQL en el servidor de forma directa y limpia en una única ventana de tiempo, evitando la complejidad técnica de mantener sincronizados dos entornos en paralelo

**Lista de Referencias**

*Si aplica, utilizar con normas APA*